

Collections in



C#.NET

Part-1

Mr. Bangar Raju



Understanding Collections in C#: A Comprehensive Guide

By Albert Harmon

Published May 1, 2017

🕒 4 min read

Technology.



TLDR This blog post explores the concept of collections in C#, highlighting their advantages over traditional arrays, including dynamic resizing, the ability to insert and delete elements, and an

overview of various collection types available in .NET.

In the world of programming, managing data efficiently is crucial. In C#, collections provide a powerful way to handle groups of objects dynamically. This blog post will delve into what collections are, how they differ from arrays, and the various types of collections available in C#.

What are Collections?

Collections in C# are dynamic data structures that allow you to store multiple values. Unlike arrays, which have a fixed size, collections can grow and shrink as needed. This flexibility makes collections a preferred choice for many programming scenarios.

The Concept of Dynamic Arrays

To understand collections, it's essential to first grasp the limitations of traditional arrays:

- **Fixed Size:** Once an array is declared, its size cannot be changed. For example, if you declare an integer array of size 10, it will always remain 10 unless you manually create a new array and copy the values.
- **Insertion and Deletion Limitations:** You cannot insert or delete elements from the middle of an array. If you need to add an

element, you must create a new array and copy the existing elements, which can be cumbersome.

The Advantages of Collections

Collections address the limitations of arrays by offering several key features:

1. **Auto Resizing:** Collections automatically adjust their size as you add or remove elements. This means you do not need to worry about the initial size or manually resizing the collection.
2. **Insertion and Deletion:** You can easily insert or delete elements at any position within a collection, making it much more flexible than arrays.
3. **Dynamic Management:** Collections manage memory more efficiently, allowing for better performance in applications that require frequent changes to data.

Types of Collections in C#

C# provides a variety of collection types, each designed for specific use cases. The collections introduced in the .NET Framework 1.0 are known as non-generic collections. Here are some common types:

- **ArrayList:** A resizable array that can hold items of any type. It is one of the most commonly used collection types.

- **Stack:** A last-in, first-out (LIFO) collection that allows you to add and remove items from the top.
- **Queue:** A first-in, first-out (FIFO) collection that allows you to add items to the end and remove them from the front.
- **LinkedList:** A collection that consists of nodes, where each node contains a value and a reference to the next node, allowing for efficient insertions and deletions.
- **Hashtable:** A collection that stores key-value pairs, allowing for fast retrieval based on keys.

Key Differences Between Arrays and ArrayLists

When comparing arrays and ArrayLists, there are three main differences:

1. **Size:** Arrays have a fixed size, while ArrayLists are variable in size and can grow as needed.
2. **Insertion:** You cannot insert items into the middle of an array, but you can easily do so in an ArrayList.
3. **Deletion:** Arrays do not allow for the removal of items from the middle, whereas ArrayLists provide methods to remove items from any position.

Working with ArrayLists

To use an ArrayList in your C# code, you need to include the ``System.Collections`` namespace. Here's a simple example of how to create and manipulate an ArrayList:

```
using System.Collections;

ArrayList myArrayList = new ArrayList();
myArrayList.Add(100);
myArrayList.Add(200);
myArrayList.Add(300);

// Displaying the capacity
Console.WriteLine("Capacity: " + myArrayList.Capacity);

// Inserting an item
myArrayList.Insert(2, 150);

// Removing an item
myArrayList.Remove(200);
```

Auto Resizing in Action

When you add items to an ArrayList, it automatically resizes itself. For instance, if you start with an empty ArrayList and add items, the initial capacity may be 0, but it will quickly adjust to accommodate the new items. The capacity typically doubles when the current capacity is reached, ensuring efficient memory usage.

Conclusion

Collections in C# provide a robust alternative to traditional arrays, offering dynamic resizing, easy insertion and deletion, and a variety of collection types to suit different needs. Understanding how to effectively use collections can significantly enhance your programming capabilities in C#. For more insights and tutorials, stay tuned for future posts.